Below is a **master table of Spring Framework terminologies** used across **all Spring versions (Spring 1.x → Spring 6.x / Spring Boot era)** with **detailed meaning, where used, real-time use cases, and examples**.

✅ I'm including **Core Spring + Spring MVC + AOP + Data + Transactions + Security + Boot + Cloud related Spring terminology**, because in real projects they are used together.

✅ Spring Framework Terminology Master Table (All Versions) + Real-time Use Cases + Examples

Legend:
Spring Core = DI, IoC, Beans
Spring MVC = controllers, web apps
Spring Data = repositories, JPA
Spring Boot = auto config
Spring Security = auth/authz
Spring Cloud = microservices

---

1) Core Spring / IoC / DI Terminology

Terminology	Meaning (Detailed)	Used In	Real-time Use Case	Example
Spring Framework	Java framework for enterprise apps (IoC + DI + AOP + Modules)	All	Building scalable backend apps	Banking / ERP / E-commerce
IoC (Inversion of Control)	Control of object creation moved from developer to container	Core	No new, container manages dependencies	Service injected in Controller
DI (Dependency Injection)	Dependencies injected automatically (constructor/setter/field)	Core	Loose coupling	UserService injected into UserController
Spring Container	Runtime environment that manages beans	Core	Handles lifecycle, injection	ApplicationContext
Bean	An object managed by Spring container	Core	Reuse of common objects	@Service, @Component beans

Bean Factory	Minimal IoC container (lazy)	Core	Lightweight container	BeanFactory usage
ApplicationContext	Advanced IoC container (eager, events, i18n)	Core	Main Spring container	AnnotationConfigApplicationContext
Bean Definition	Metadata describing how to create/configure a bean	Core	Used in configuration	XML/Annotation metadata
Bean Scope	Bean lifecycle boundary	Core	Correct lifecycle	Singleton for services, Prototype for temp objects
Singleton Scope	One bean per container	Core	Shared objects	Services
Prototype Scope	New bean per request	Core	Per operation objects	Parser/Builder objects
Request Scope	One bean per HTTP request	Web	Request-specific data	RequestContext bean
Session Scope	One bean per HTTP session	Web	User session state	Shopping cart
GlobalSession Scope	For Portlets (legacy)	Web	Old portal systems	Enterprise portal
Bean Lifecycle	Steps: instantiate → inject → init → ready → destroy	Core	Resource management	DB connection close
init-method	Called after bean creation	Core	Initialize custom logic	init() method
destroy-method	Called before bean destruction	Core	Cleanup resources	close file/db
@PostConstruct	Init hook (JSR-250)	Core	Setup resources	load cache
@PreDestroy	Cleanup hook (JSR-250)	Core	Release resources	stop threads
Aware Interfaces	Allows bean to know Spring internals	Core	Advanced control	ApplicationContextAware
BeanPostProce	Hook to modify beans	Core	Proxy	Used internally for

ssor	before/after init		creation, annotation processing	@Autowired
FactoryBean	A bean that produces other beans	Core	Complex object creation	Creating Hibernate SessionFactory
@Component	Generic bean marker	Core	Auto-scan and register	@Component class EmailUtil
@Service	Bean marker for service layer	Core	Business logic	@Service class PaymentService
@Repository	DAO marker + exception translation	Data/Core	DB logic	@Repository class UserDao
@Controller	MVC controller marker	Web	Handling pages	MVC apps
@RestController	Controller + @ResponseBody	Web	REST APIs	JSON APIs
@Configuration	Config class for beans	Core	Java config	@Bean methods
@Bean	Declares bean inside config class	Core	Manual bean creation	@Bean public ObjectMapper
Component Scanning	Auto-detect beans in packages	Core	Reduces XML config	@ComponentScan("com.app")
@Autowired	Inject dependency automatically	Core	DI injection	@Autowired UserService
Constructor Injection	DI through constructor	Core	Best practice, immutable	public UserController(UserService s)
Setter Injection	DI through setters	Core	Optional dependencies	setEmailService()
Field Injection	DI via fields	Core	Quick but discouraged	@Autowired private Repo repo;
@Qualifier	Select specific bean name	Core	Multiple implementations	@Qualifier("sms") NotificationService
@Primary	Default bean when	Core	Preferred	Primary Repo

	multiple exist		implementation	
@Lazy	Delay bean creation until used	Core	Reduce startup time	heavy beans in microservices
@DependsOn	Force bean initialization order	Core	Order control	Cache init before service
Spring Profiles	Environment based configuration	Core/Boot	dev/test/prod config	@Profile("prod")
@PropertySource	Load external properties	Core	Config externalization	load app.properties
Environment	Access runtime properties and profiles	Core	dynamic config	env.getProperty("key")
Property Placeholder	Replace ${...} values	Core	config injection	${server.port}

---

2) AOP Terminology (Cross-cutting concerns)

Terminology	**Meaning**	**Used In**	**Real-time Use Case**	**Example**
AOP	Aspect Oriented Programming	AOP	Logging/security/transactions	Log execution time
Cross-cutting Concern	Feature applied across modules	AOP	Logging across all services	audit logging
Aspect	Modular AOP logic	AOP	Define logging aspect	@Aspect class LoggingAspect
Advice	Code executed at joinpoint	AOP	Before/After/Around	Around advice for timing
Join Point	Point in program execution	AOP	method execution	service method call
Pointcut	Expression selecting joinpoints	AOP	select methods	execution(* com.app..*(..))

Proxy	Wrapper object created by Spring	AOP	Apply aspects	Transaction proxy
Weaving	Applying aspects to code	AOP	runtime weaving	Spring uses proxy-based weaving
@Before	Advice before method call	AOP	validate inputs	check auth
@After	Advice after method ends	AOP	cleanup	release resources
@AfterReturning	After successful completion	AOP	auditing	log success
@AfterThrowing	On exception	AOP	error logging	log failures
@Around	Wrap method execution	AOP	timing and retries	latency measurement

---

3) Spring MVC Terminology (Web Layer)

Terminology	**Meaning**	**Used In**	**Real-time Use Case**	**Example**
DispatcherServlet	Front controller of Spring MVC	MVC	Routes request → controller	REST gateway
Front Controller Pattern	Central request handling entrypoint	MVC	uniform routing	DispatcherServlet
Controller	Handles web requests	MVC	API endpoints	@GetMapping("/users")
HandlerMapping	Maps URL to controller method	MVC	URL mapping	RequestMappingHandlerMapping
HandlerAdapter	Executes handler/controller	MVC	method execution	MVC internal
ViewResolver	Resolves view templates	MVC	JSP/Thymeleaf views	InternalResourceViewResolver

Model	Data passed to view	MVC	MVC pages	model.addAttribute()
@RequestMapping	Generic request mapping	MVC	route mapping	@RequestMapping("/api")
@GetMapping	GET route mapping	MVC	fetch data	/users
@PostMapping	POST route mapping	MVC	create resource	create order
@PutMapping	PUT route mapping	MVC	update resource	update user
@DeleteMapping	DELETE route mapping	MVC	delete	delete user
@PathVariable	Take value from URL path	MVC	dynamic urls	/users/{id}
@RequestParam	Take query param	MVC	filtering	/users?page=1
@RequestBody	Read JSON request	MVC	REST API input	create user payload
@ResponseBody	Send response body (JSON)	MVC	REST API output	return DTO
@ResponseStatus	Force HTTP status code	MVC	correct status	201 created
HttpMessageConverter	Converts JSON↔Java	MVC	Serialization	Jackson converter
Interceptor	Pre/post request hooks	MVC	auth logging	HandlerInterceptor
Filter	Servlet filter before Spring	Web	CORS, security	JWT filter
Exception Handling	Global handler for errors	MVC	consistent errors	@ControllerAdvice
@ControllerAdvice	Global exception handler	MVC	centralized exception mapping	API error response
@ExceptionHandler	Handler for exceptions	MVC	custom handling	handle UserNotFoundException

---

4) Spring Transaction Management Terminology

Terminology	Meaning	Used In	Real-time Use Case	Example
Transaction	Group of DB operations atomic	TX	bank transfer	debit + credit
ACID	Atomicity Consistency Isolation Durability	TX	safe DB operations	payment systems
@Transactional	Declarative transaction	TX	automatic rollback	order placement
Propagation	Defines how TX behaves with nesting	TX	nested service calls	REQUIRED/REQUIRES_NEW
REQUIRED	Join existing tx else create	TX	standard	default
REQUIRES_NEW	Create new tx always	TX	audit logging separate	audit even if main fails
Isolation	Handles concurrency reads/writes	TX	prevent dirty reads	SERIALIZABLE
Rollback	undo operations on error	TX	safe operations	rollback on exception
Checked Exception	not rolled back by default	TX	explicit rollback	rollbackFor=Exception.class

---

5) Spring Data / ORM Terminology

Terminology	Meaning	Used In	Use Case	Example
ORM	Map Java objects to DB tables	JPA/Data	DB persistence	User entity
Entity	JPA mapped class	Data JPA	DB table mapping	@Entity User
Repository	Data access abstraction	Spring Data	CRUD without DAO	UserRepository extends JpaRepository

JpaRepository	JPA repository with CRUD + paging	Data JPA	fast DB operations	findAll(Pageable)
PagingAndSortingRepository	paging and sorting support	Data	large datasets	pagination
CrudRepository	basic CRUD operations	Data	small apps	save, findById
Derived Query	query from method name	Data	no SQL writing	findByEmail()
JPQL	Query language for entities	JPA	custom query	@Query("select u from User u")
Native Query	raw SQL execution	JPA	performance tuning	nativeQuery=true
Lazy Loading	Load relation when needed	ORM	reduce DB load	orders load on demand
Eager Loading	Load relation immediately	ORM	small relations	user roles
N+1 problem	ORM loads too many queries	ORM	performance issue	fixed using fetch join

---

6) Spring Boot Terminology (Modern Spring)

Terminology	**Meaning**	**Used In**	**Real-time Use Case**	**Example**
Spring Boot	Opinionated Spring starter framework	Boot	rapid microservices dev	REST microservice
Auto Configuration	Boot configures beans automatically	Boot	no manual setup	auto DataSource
Starter	dependency bundles	Boot	quick setup	spring-boot-starter-web
Actuator	production monitoring endpoints	Boot	health metrics	/actuator/health
Externalized Config	config via properties/yml/env	Boot	dev/prod separation	application-prod.yml

	vars			
Embedded Server	Tomcat/Jetty inside jar	Boot	run jar directly	java -jar app.jar
Fat Jar	jar including dependencies	Boot	deployment easy	jar deploy in VM
@SpringBootApplication	Main boot annotation	Boot	enable auto config	main class
@EnableAutoConfiguration	enable auto config	Boot	boot magic	inside @SpringBootApplication
@ConfigurationProperties	bind property values to class	Boot	structured config	DB properties class
CommandLineRunner	run startup logic	Boot	seed data	preload users

---

7) Spring Security Terminology

Terminology	Meaning	Used In	Use Case	Example
Authentication	verify who user is	Security	login	JWT login
Authorization	verify access rights	Security	role-based access	ADMIN role
Principal	authenticated user object	Security	get logged user	SecurityContextHolder.getContext()
SecurityContext	holds authentication info	Security	session tracking	per request
FilterChain	security filters pipeline	Security	JWT filter	verify token
CSRF	cross-site request forgery protection	Security	web apps	enabled in forms
JWT	token based auth	Security	stateless microservices	bearer token

OAuth2	delegated auth	Security	login with Google	OAuth2 login
RBAC	role based access control	Security	admin/user features	hasRole('ADMIN')

---

8) Spring Cloud / Microservices Terminology

Terminology	Meaning	Used In	Use Case	Example
Microservices	apps split into services	Cloud	scalable enterprise	Order + Payment service
Service Discovery	register services + locate dynamically	Cloud	dynamic routing	Eureka
Config Server	centralized config	Cloud	shared configuration	Spring Cloud Config
API Gateway	entry point for services	Cloud	routing + security	Spring Cloud Gateway
Circuit Breaker	stop cascading failures	Cloud	resilience	Resilience4j
Load Balancing	distribute traffic	Cloud	scaling	Spring Cloud LoadBalancer
Distributed Tracing	trace request across services	Cloud	debugging	Zipkin
Observability	logs + metrics + traces	Cloud	production monitoring	Actuator + Micrometer

---

✅ Real-world Example Map (Terminology → Scenario)

Scenario	Terminologies Used
E-commerce order placement	@RestController, @Service, @Transactional, Repository, DTO, Validation
Banking transfer	@Transactional, Isolation, Propagation, AOP logging
Login & access control	Spring Security, JWT, FilterChain, Authorization

Microservices deployment	Spring Boot, Cloud Gateway, Config Server, Circuit Breaker
Large dataset processing	Pagination, JpaRepository, Batch Processing
Performance improvement	Lazy Loading, N+1 fix, caching

If you want, I can generate a **2nd mega table** containing **Spring XML terminologies (Spring 1.x–3.x legacy)** like:

- bean, property, constructor-arg, context:component-scan, aop:config
- tx:annotation-driven
- mvc:annotation-driven
- FactoryBean, ProxyFactoryBean, BeanNameAutoProxyCreator

…and their modern equivalents in Spring 6 / Spring Boot.